Sieci Neuronowe
Laboratorium 01
Wstęp do Pythona, Cześć 1

Andrzej Kordecki

Zakład Teorii Maszyn i Robotów
Instytut Techniki Lotniczej i Mechaniki Stosowanej
Wydział Mechaniczny Energetyki i Lotnictwa
Politechnika Warszawska
2026

1 Python

Python jest wysokopoziomowym językiem programowania ogólnego przeznaczenia. Charakteryzuje się stosunkowo prostą i czytelną składnią oraz bardzo dużą liczbą dostępnych bibliotek i pakietów, dzięki czemu jest szeroko wykorzystywany między innymi w analizie danych, obliczeniach naukowych, automatyzacji, uczeniu maszynowym oraz sztucznej inteligencji. Python został stworzony przez Guido van Rossuma. Prace nad pierwszą implementacją języka rozpoczęły się pod koniec 1989 roku, a jego pierwsza publiczna wersja została udostępniona w 1991 roku. Jednym z podstawowych założeń przy projektowaniu języka była czytelność i przejrzystość kodu źródłowego. Zasady te są nadal widoczne zarówno w składni języka, jak i w sposobie jego dalszego rozwoju. Python jest rozwijany jako projekt Open Source, a rozwój języka i jego społeczności wspierany jest przez Python Software Foundation. Oficjalna strona projektu, zawierająca dokumentację, informacje o języku oraz możliwość pobrania interpretera Python, dostępna jest pod adresem:

`https://www.python.org`

1.1 Python jest językiem interpretowanym

Python jest dynamicznie typowanym językiem programowania, który najczęściej wykonywany jest za pomocą interpretera. Oznacza to, że kod źródłowy nie musi być wcześniej ręcznie kompilowany do osobnego programu wykonywalnego. Interpreter analizuje kod programu i wykonuje zawarte w nim instrukcje. W języku Python typ zmiennej jest określany na podstawie przypisanej do niej wartości i może zmieniać się w trakcie wykonywania programu. Przykładowo:

```
x = 10
print(type(x))

x = 3.14
print(type(x))
```

W pierwszym przypadku zmienna "x"przechowuje liczbę całkowitą, natomiast po wykonaniu kolejnego przypisania przechowuje liczbę zmiennoprzecinkową. Podczas zajęć laboratoryjnych wykorzystywane będzie środowisko Google Colab. Jest to działające w przeglądarce internetowej środowisko typu Jupyter Notebook, które umożliwia pisanie i wykonywanie kodu Python w osobnych komórkach. Kod jest wykonywany w zdalnym środowisku uruchomieniowym, dzięki czemu do rozpoczęcia pracy nie jest wymagane lokalne instalowanie interpretera Pythona ani większości używanych podczas zajęć bibliotek. Program w Colabie może być podzielony na kolejne komórki zawierające kod oraz tekst opisowy. Ułatwia to wykonywanie programu krok po kroku, obserwowanie wyników poszczególnych operacji oraz eksperymentowanie z kodem. Z tego względu środowiska notebookowe są często wykorzystywane w analizie danych, uczeniu maszynowym i podczas prototypowania modeli sieci neuronowych.

Alternatywą dla języków interpretowanych są języki, w których kod źródłowy jest przed uruchomieniem kompilowany, np. C lub C++. W takim przypadku kompilator przekształca kod źródłowy do postaci programu

wykonywalnego. Podział na języki interpretowane i kompilowane jest jednak pewnym uproszczeniem, ponieważ współczesne środowiska wykonawcze mogą łączyć oba mechanizmy.

Dokumentacja języka Python dostępna jest na oficjalnej stronie:

`https://docs.python.org/3/`

Bardziej obszerne wprowadzenie do programowania w języku Python oraz zagadnień wykorzystywanych podczas laboratoriów znajduje się również w materiałach dodatkowych udostępnionych w zespole przedmiotu.

2 Typy danych i zmiennych

Język programowania Python jest dynamicznie typowany, co oznacza, że typy danych są określane automatycznie podczas wykonywania programu, a nie na etapie deklaracji zmiennych. Python zawiera wiele typów danych, które można podzielić na następujące kategorie:

- Liczbowe: Booleans (wartości logiczne), Integers (liczby całkowite), Floats (liczby zmiennoprzecinkowe), Complex (liczby zespolone),
- Sekwencje: Strings (napisy), Lists (listy), Tuples (krotki),
- Odwzorowujące: Dictionaries (słowniki), Sets (zbiory),
- NoneType: Reprezentuje brak wartości lub pustą zmienną,
- Obiekty użytkownika: Klasy (custom objects).

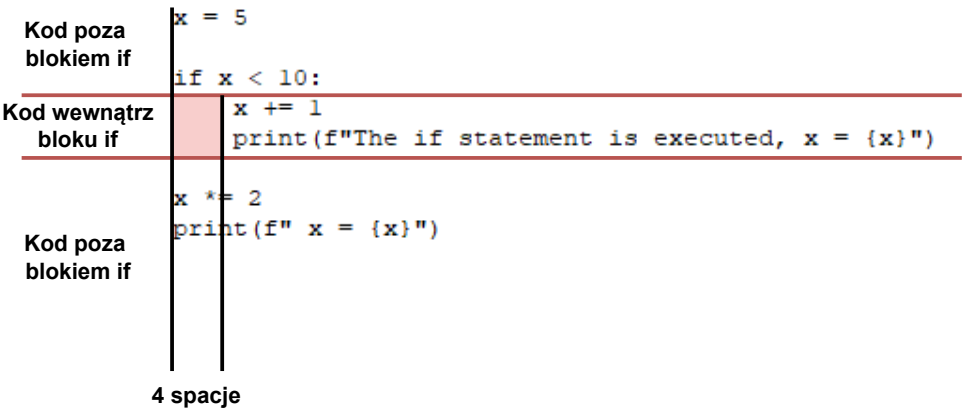
Należy pamiętać, że w różnych wersjach Pythona następowały zmiany w klasyfikacji niektórych typów danych.

3 Narzędzia kontroli przepływu

Interpreter Pythona wykonuje kod programu wiersz po wierszu, ale możemy to zmienić za pomocą narzędzi kontroli przepływu. Narzędzia te wpływają na kolejność wykonywania kodu w programie. Przykład najczęściej używanych narzędzi przepływu: 'if', 'for', 'while', 'def'. Definicje funkcji, pętli i instrukcji warunkowych mają podobną składnię i wykorzystują wcięcia do wykonywania przypisanego im bloku kodu programu poprzedzonych znakiem dwukropka.

Python używa **białych znaków** (4 spacji w PEP 8) do wyznaczenia zasięgu działania instrukcji.

Instrukcje mogą być zagnieżdżone na kilka poziomów poprzez wykorzystanie wielokrotności 4 spacji. Przykład bloku instrukcji „if" po kazano na Rys. 1. Odstępy w liniach kodu mają duże znaczenie w jego interpretacji.



Rysunek 1: Położenie instrukcji zagnieżdżony w bloku instrukcji „if"

Dlatego podkreślam, NIE używamy jak w innych językach programowania nawiasów lub słów, ale białych znaków do wyznaczenia zasięgu działania instrukcji.

3.1 Klasy

Programowanie obiektowe (Object-oriented programming, OOP) łączy grupę atrybutów (danych) z funkcjami lub metodami w jednostkę zwaną "obiektem". Napisany obiektowo program zawiera zbiór obiektów, komunikujących się pomiędzy sobą w celu wykonywania zadań. Podejście to różni się od tradycyjnego programowania proceduralnego, gdzie dane i procedury nie są ze sobą bezpośrednio związane. Podstawowe pojęcia OOP:

- Klasa – zawiera definicje formatu danych oraz procedur dla danego obiektu.
- Obiekt – instancja klasy.

Każdy obiekt jest instancją konkretnej klasy np. obiekt 'pies' może być instancją klasy 'zwierzęta'. Procedury w programowaniu obiektowym nazywane są metodami, a zmienne: obszarami, członkami, atrybutami albo właściwościami. Programowanie obiektowe zapewnia warstwę abstrakcji, która może być użyta do odseparowania wewnętrznego od zewnętrznego kodu programu. Obiekty są wywoływane przez specjalne typy metod w klasie znanych jako konstruktory. Program może stworzyć wiele instancji tej samej klasy w czasie działania, które działają niezależnie od siebie.

3.2 Formatowanie kodu w standardzie PEP 8

Kod programu można napisać w różnych stylach, które pozwalają na uzyskanie programu łatwego w odczycie i analizie jego wykonywania. Standardowym stylem języka Python, a więc regułami i zaleceniami jakie zachowujemy przy pisaniu kodu programu, jest styl PEP 8. Przykładowe zalecenia PEP 8:

- Wcięcia kodu powinny być wykonane 4 spacjami bez stosowania tabulatorów. Wielkość 4 spacji jest dobrym kompromisem między małym wcięciem (pozwala na większą głębokość zagnieżdżenia), a dużym wcięciem (łatwiejszym do czytania).
- Zawijaj wiersze tak, aby nie przekraczały 79 znaków. Pomaga to wyświetlić program użytkownikom z małymi wyświetlaczami i umożliwia umieszczenie kilku plików kodu obok siebie na większych wyświetlaczach.
- Użyj pustych linii do oddzielenia funkcji i klas oraz większych bloków kodu wewnątrz funkcji.
- Jeśli to możliwe, wstawiaj komentarze w osobnych wierszach.
- Używaj spacji wokół operatorów i po przecinkach, ale nie bezpośrednio wewnątrz konstrukcji nawiasów: a = f(1, 2) + g(3, 4).
- Klasy i funkcje powinny mieć spójne nazewnictwo. Konwencją jest rozpoczynanie klas wielką literą, a funkcji małą literą.

Więcej informacji na stronie: `https://www.python.org/dev/peps/pep-0008/`

4 Biblioteki Pythona

Biblioteki Pythona rozszerzają możliwości języka o dodatkowe funkcje, klasy, typy danych oraz gotowe algorytmy, które można wykorzystać podczas tworzenia programów. Biblioteka może składać się z jednego lub wielu modułów i pakietów. W trakcie laboratoriów mogą być wykorzystywane między innymi następujące biblioteki:

- NumPy – podstawowa biblioteka do obliczeń numerycznych i naukowych. Umożliwia między innymi wykonywanie operacji na wektorach, macierzach i tablicach wielowymiarowych.
- Matplotlib – biblioteka służąca do tworzenia wykresów i wizualizacji danych. Umożliwia szeroką konfigurację wyglądu wykresów, ich osi, opisów oraz sposobu prezentacji danych.
- OpenCV (cv2) – biblioteka przeznaczona głównie do przetwarzania obrazów i zastosowań z zakresu wizji komputerowej. Zawiera między innymi funkcje do wczytywania i zapisywania obrazów, zmiany przestrzeni barw, filtrowania, wykrywania krawędzi oraz wykonywania operacji geometrycznych.
- scikit-learn (sklearn) – biblioteka zawierająca narzędzia do klasycznego uczenia maszynowego oraz analizy danych. Udostępnia między innymi algorytmy klasyfikacji, regresji, grupowania danych, redukcji wymiarowości, wstępnego przetwarzania danych oraz obliczania metryk jakości modeli.

- TensorFlow – jedna z głównych bibliotek przeznaczonych do tworzenia i trenowania modeli uczenia maszynowego oraz głębokich sieci neuronowych.
- PyTorch – jedna z głównych bibliotek wykorzystywanych do tworzenia i trenowania modeli uczenia maszynowego oraz głębokich sieci neuronowych. Jest szeroko stosowana zarówno w badaniach naukowych, jak i w praktycznych zastosowaniach.
- Ultralytics – biblioteka udostępniająca gotowe modele i narzędzia przeznaczone głównie do zadań wizji komputerowej. Jest szczególnie znana z implementacji modeli z rodziny YOLO, wykorzystywanych między innymi do detekcji obiektów, segmentacji oraz estymacji pozy.

Bibliotekę lub moduł można zaimportować do programu za pomocą słowa kluczowego "import". Nazwa używana podczas importowania może różnić się od nazwy pakietu użytej podczas jego instalacji. Jeżeli nazwa biblioteki jest długa lub często używana, można przypisać jej krótszy alias za pomocą słowa kluczowego "as":

```
import numpy as np

a = np.sqrt(4)
print(a)  # Wynik: 2.0
```

Możliwe jest również zaimportowanie wybranego elementu biblioteki za pomocą słowa kluczowego "from":

```
from numpy import sqrt

a = sqrt(4)
print(a)  # Wynik: 2.0
```

5 Google Colaboratory

Laboratorium będzie prowadzone z wykorzystaniem języka Python na platformie Google Colaboratory (w skrócie Colab). Colab pozwala każdemu pisać i wykonywać dowolny kod Pythona za pomocą przeglądarki internetowej, a dokładnie to serwer i interpreter Jupyter notebook. Strona projektu Colab:

`https://colab.research.google.com`.

Platforma Colab nie wymaga instalacji ani konfiguracji interpretera na komputerze, a wszystkie niezbędne biblioteki do wykorzystania podczas laboratorium są już wstępnie zainstalowane. Ponadto Colab zapewnia bezpłatny dostęp do zasobów obliczeniowych, które są przydzielane dynamicznie w zależności od napisanego projektu.

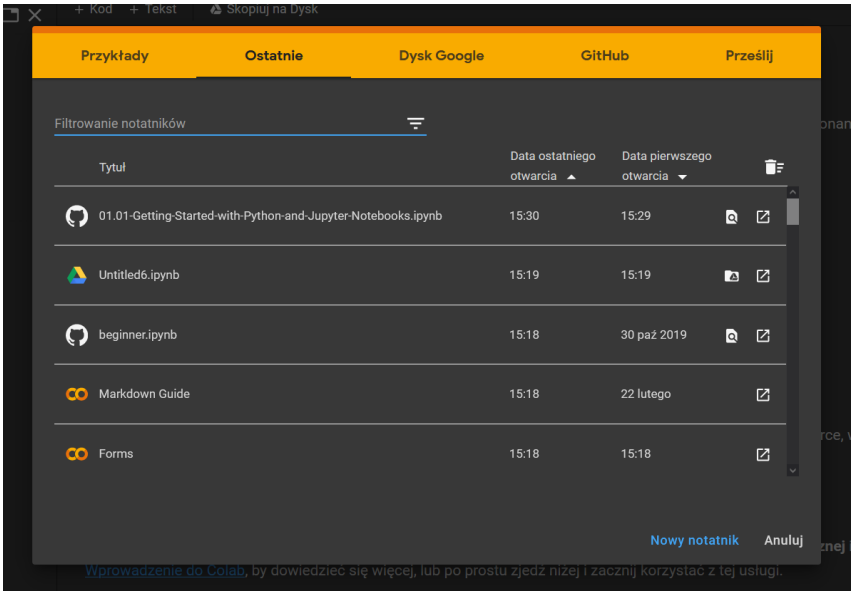
Aby korzystać z Colaba, należy zalogować się za pomocą konta Google. Ze względów bezpieczeństwa zalecam utworzenie nowego konta, które będzie wykorzystywane wyłącznie podczas laboratoriów. Rekomendowane jest również korzystanie z trybu „prywatnego” w przeglądarce, który automatycznie wylogowuje z Colaba po jej zamknięciu.

6 Nowy notatnik i uruchomienie programu w Colabie

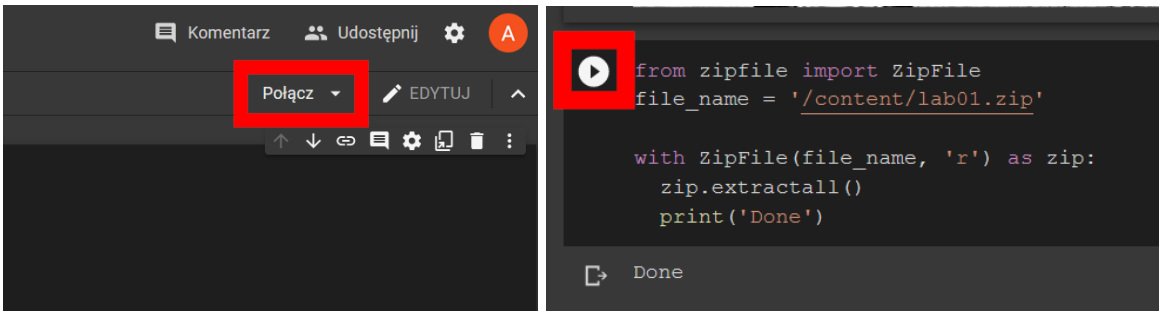
Po zalogowaniu się na stronie projektu Colaba otworzy się okno, które umożliwi wybór wcześniej pisanego lub utworzenie nowego notatnika (Rys. 2).

Nowy notatnik możemy utworzyć z istniejącego notatnika przez wybór zakładki 'Plik' i następnie opcji 'Nowy notatnik'. Utworzony nowy projekt nie jest statyczną stroną internetową tylko środowiskiem interaktywnym notatnika Colaba umożliwiającym pisanie i uruchamianie kodu. Uruchamiania programu i wyświetlanie wyników notatnika jest zgodne z obsługą notatników Jupyter Notebook. Kod zazwyczaj będzie pisany i uruchamiany w osobnych blokach (komórkach). W celu uruchomienia kodu programu należy w Colabie w pierwszej kolejności połączyć się z serwer w celu przydzielenia zasobów sprzętowych, a następnie przycisku wykonania kodu programu (Rys. 3).

Napisany kod programu jest automatycznie zapisywany, ale istnieje możliwość jego ściągnięcia na dysk twardy komputera lub zapisanie w innej lokalizacji dysku sieciowego Google'a. Opcja automatycznego zapisywania powoduje również, że nie musimy obawiać się przypadkowego zamknięcia programu, który po ponownym wejściu na stronę Colaba będzie dla nas dostępny w takiej samej postaci. Więcej informacji na temat pisania programu w notatnikach Jupyter Notebook zamieszczono na stronie: link



Rysunek 2: Okno menadżera projektów



Rysunek 3: Lokalizacja przycisku 'Połącz' (po lewej) oraz 'Uruchom' komórkę z kodem (po prawej)

6.1 Wyszukiwanie ścieżki plików w folderach

Podczas laboratorium możemy mieć potrzebę z zapisaniem lub wczytaniem pliku. Stąd, niezbędne będzie określenie miejsca przechowywania pliku, czy folderu. Platforma Colab umożliwia łatwe odczytanie ścieżki do pliku z widoku menadżera plików projektu. W tym celu należy wybrać polecenie 'skopiuj ścieżkę' z rozwijanego menu pliku po kliknięciu na wybranym pliku prawym przyciskiem myszy (Rys. 4).

Nie zawsze chcemy operować tylko na jednym pliku w programie, a zazwyczaj chcemy wykonać operacje na wszystkich plikach w danym folderze. Kod programu do wyszukiwania wszystkich plików w folderze bez konieczności ręcznego wpisywania nazwy każdego pliku:

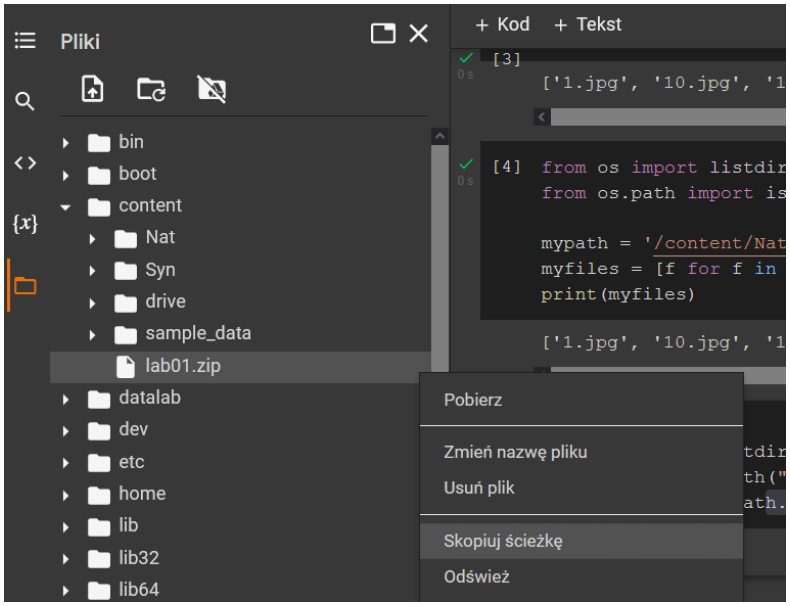
```
from os import listdir
from os.path import isfile, join

mypath = '/content/n'
myfiles = [f for f in listdir(mypath) if isfile(join(mypath, f))]
print(myfiles)
```

Wyszukiwanie plików można zapisać jako funkcję i sprecyzować obszar szukania do plików z określonym formatem:

```
search_jpg = lambda a, b: [a + '/'+ f for f in listdir(a) if f.endswith(b)]
mypath = '/content/Nat'
fextension = '.jpg'
myfiles = search_jpg(mypath, fextension)
print(myfiles)
```

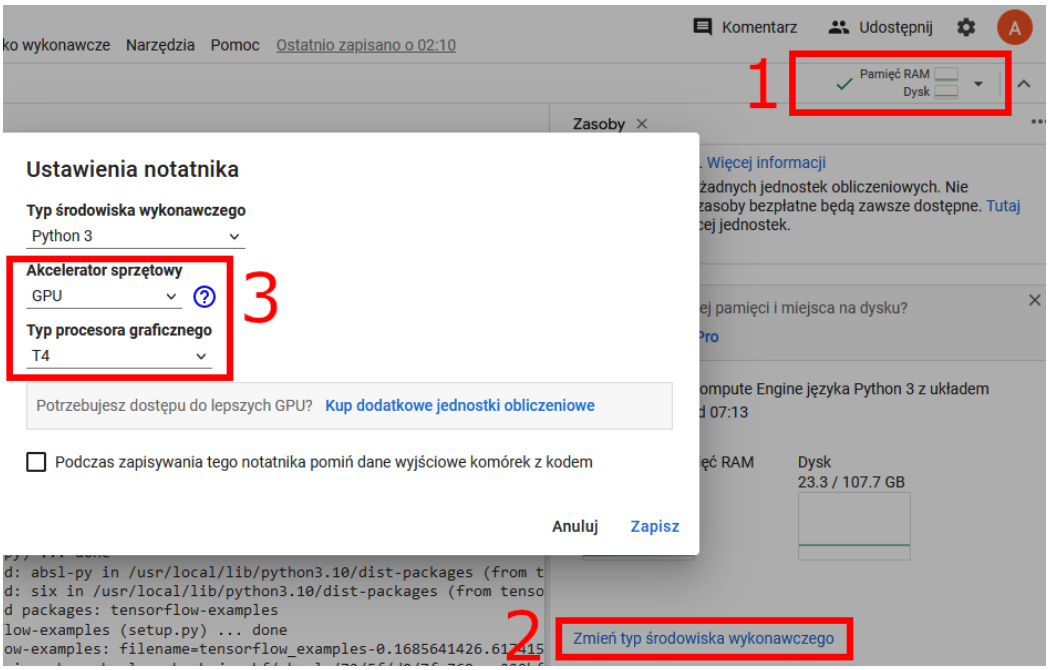
Funkcja zwróci listę ze ścieżkami do wszystkich plików w danym folderze.



Rysunek 4: Położenie przycisku do odczytu ścieżki pliku w notatniku Colab

7 Akceleracja GPU obliczeń w Colabie

Google umożliwia korzystanie z bezpłatnego procesora graficznego T4 do akceleracji obliczeń w notebookach Colab. Zmiana na GPU powinna być dokonana, gdy czas obliczeń staje się zbyt długi, co będzie miało miejsce głównie w ostatnich laboratoriach. Niestety, włączenie akceleracji nie zawsze jest możliwe i zależy od dostępnych mocy obliczeniowych Google. Aby włączyć GPU, należy wybrać odpowiednie opcje z menu zgodnie z rysunkiem 5.



Rysunek 5: Wybierz następujące opcje w celu uruchomienia akceleracji obliczeń GPU T4 w Colabie

8 Zadania do rozwiązania

Na końcu instrukcji znajdują się zadania, których celem jest zapoznanie się z najważniejszymi elementami programowania w języku Python oraz przećwiczenie ich praktycznego zastosowania. Zadania obejmują między innymi pracę z funkcjami, kolekcjami danych, instrukcjami warunkowymi, pętlami, klasami oraz wybranymi bibliotekami. Liczba przygotowanych zadań jest większa niż możliwa do wykonania podczas jednych zajęć laboratoryjnych. Nie ma więc konieczności realizowania wszystkich zadań w trakcie laboratorium. W pierwszej kolejności warto wybrać te zagadnienia, w których czujemy się mniej pewnie lub z którymi nie mieliśmy wcześniej do czynienia.

Osoby, które dopiero rozpoczynają programowanie w języku Python lub chcą przypomnieć sobie podstawowe zasady tworzenia programów, mogą skorzystać z kolejnych podrozdziałów. Przedstawiono w nich krok po kroku sposób rozwiązania dwóch pierwszych zadań wraz z wyjaśnieniem zastosowanych instrukcji i mechanizmów języka Python. Przedstawione rozwiązania są jedynie przykładowe. W programowaniu to samo zadanie można często rozwiązać na kilka różnych sposobów.

8.1 Przykład rozwiązania zadania 1 w Colabie

Celem Zadania 1 jest wykonanie konwersji temperatury ze stopni Celsjusza (°C) na stopnie Fahrenheita (°F). Podczas realizacji zadania zostaną omówione kluczowe elementy potrzebne do napisania programu w języku Python, a także podstawowe funkcje środowiska Google Colab, niezbędne do jego uruchomienia.

> **Zadanie 1:** Napisz funkcję "convert_temp", która przyjmuje jeden argument "x"reprezentujący temperaturę w stopniach Celsjusza. Funkcja powinna sprawdzić, czy "x"jest liczbą (całkowitą lub zmiennoprzecinkową). Jeśli "x"jest liczbą, funkcja powinna przekonwertować temperaturę na stopnie Fahrenheita i wypisać wynik (Uwaga: $F = 1,8 * C + 32$).

Wykonanie programu krok po kroku:

1. Uruchomienie środowiska Colab. Kliknij Nowy notatnik (New Notebook) i następnie wybierz pierwszą komórkę do edycji kodu programu. W pierwszej komórce wpisz i sprawdź poprawność działania środowiska Colab:

```
print("Zadanie 1")
```

   Uruchom komórkę naciskając "trójkąt"po lewej górnej stronie komórki lub klawisze Shift + Enter. Na dole komórki powinien pojawić się napis: "Zadanie 1".

2. Tworzenie funkcji w Pythonie. Funkcja w języku Python tworzona jest za pomocą słowa kluczowego "def", które pozwala zdefiniować blok kodu wykonujący określone zadanie. Funkcje mogą przyjmować argumenty (wejścia), zwracać wartości (wyjścia) i umożliwiają wielokrotne używanie tego samego fragmentu programu w różnych miejscach. Ogólna definicja funkcji w Pytanie oraz przykład funkcja dodającej 2 do zadanej liczby (argumentu funkcji):

```
# Budowa funkcji
def nazwa_funkcji(argumenty):
  # instrukcje
  return wynik

# Przykladowa funkcja dodajaca 2 do x
def add_two(x):
  return x + 2

print(add_two(5))
```

   Na dole komórki powinien pojawić się wynik: 7

3. Sprawdzenie typu zmiennej. W Pythonie typ zmiennej można sprawdzić na kilka sposobów. Najczęściej stosuje się funkcję "isinstance()", która przyjmuje dwa argumenty: zmienną do sprawdzenia i typ (lub krotkę typów), z którymi chcemy porównać. Funkcja "isinstance()"zwróci: "True"– jeżeli zmienna jest zgodna z podanym typem lub "False"– jeżeli zmienna ma inny typ. Drugim sposobem jest funkcja "type()", która przyjmuje jako argument badaną zmienną i zwraca jej typ danych. Funkcja "isinstance()"jest bardziej elastyczne niż "type()", ponieważ umożliwia sprawdzanie wielu typów jednocześnie (np. liczby całkowite i zmiennoprzecinkowe). W Pythonie często istnieje kilka funkcji, które mogą prowadzić do tego samego

efektu, a wybór odpowiedniego rozwiązania zależy od naszych preferencji. Na podstawie sprawdzenia typu zmiennej możemy zbudować za pomocą narzędzi kontroli przepływu "if"kod, który zostanie wykonany jeżeli nasza zmienna jest liczbą. Przykład:

```
x = 10

# Sprawdz czy jest typu calkowitego (int)
print(isinstance(x, int), type(x)==int)   # wynik: True, True
# Sprawdz czy jest typu zmiennoprzecnikowego (int)
print(isinstance(x, float), type(x)==float) # wynik: False, False

if isinstance(x, (int, float)):
  print("x jest liczba - kod do wykonania, gdy jest liczba")
else:
  print("x nie jest liczba")
```

4. Implementacja funkcji konwersji temperatury "convert_temp". Utwórzmy funkcję na podstawie wiedzy z poprzednich kroków:

```
def convert_temp(x):
  # Sprawdzenie czy x jest liczba calkowita lub zmiennoprzecinkowa
  if isinstance(x, (int, float)):
    # Obliczamy temperature w Fahrenheitach
    f = 1.8 * x + 32
    print(f"Temperatura {x}C to {f}F")
  else:
    print("Blad: podana wartosc nie jest liczba")
```

5. Testowanie funkcji. Mocno zalecane jest sprawdzenie napisanego kodu programu w celu wyszukania ewentualnych błędów w jego działaniu:

```
convert_temp(0)      # spodziewany wynik: 32F
convert_temp(25)     # spodziewany wynik: 77F
convert_temp(-40)    # spodziewany wynik: -40F
convert_temp(36.6)   # spodziewany wynik: 97.88F
convert_temp("abc") # spodziewany wynik: komunikat o bledzie
```

6. Wygląd kodu programu w środowisku Colab zastał przedstawiony na rysunku 6.

8.2 Przykład rozwiązania zadania 2 w Colabie

Celem Zadania 2 jest zapoznaniem się typami danych i praca nad kolekcjami elementów w celu usuwania z nich duplikatów.

Zadanie 2: Zdefiniuj funkcję o nazwie "usun_duplikaty", która ma jeden argument o nazwie "sekwencja"w postaci listy liczb całkowitych. Funkcja powinna wyznaczyć unikalne wartości występujące na liście, uporządkować je rosnąco, a następnie wypisać, ile razy każda z nich występuje w liście wejściowej.

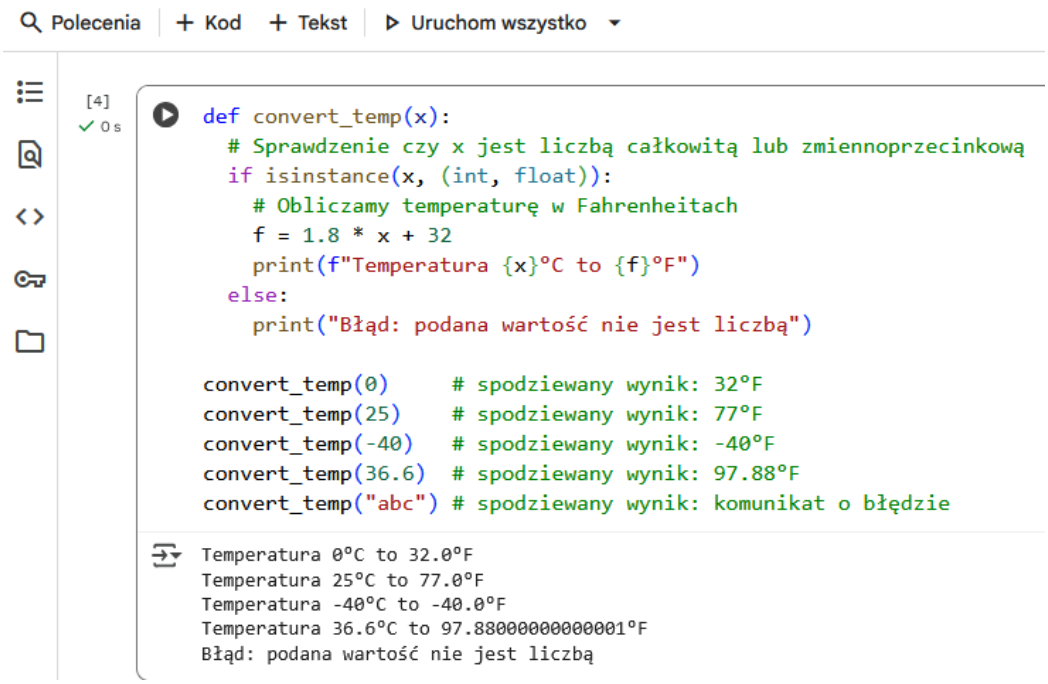
Wykonanie programu krok po kroku:

1. Na początku definiujemy funkcję "usun_duplikaty"przy użyciu słowa kluczowego "def". Funkcja przyjmuje jeden parametr wejściowy o nazwie "sekwencja".

```
def usun_duplikaty(sekwencja):
  pass
```

Instrukcja "pass"nie wykonuje żadnej operacji. Może być wykorzystana jako instrukcja tymczasowa, która pozwala utworzyć pustą funkcję bez błędu interpretera. W kolejnych krokach zostanie ona zastąpiona właściwymi instrukcjami.

2. Aby usunąć duplikaty z listy, można wykorzystać typ danych "set", który przechowuje wyłącznie unikalne elementy. Przekształcenie listy w zbiór powoduje automatyczne usunięcie powtarzających się wartości. Należy jednak pamiętać, że typ "set"nie zachowuje kolejności elementów oraz nie umożliwia odwoływania się do nich za pomocą indeksów, ponieważ ma strukturę nieuporządkowaną.



Rysunek 6: Kod programu realizującego zadanie 1

```
unikalne = set(sekwencja)
```

3. Aby usunąć duplikaty z listy, można wykorzystać typ danych "set", reprezentujący zbiór. Zbiór przechowuje wyłącznie unikalne elementy, dlatego przekształcenie listy w zbiór powoduje automatyczne usunięcie powtarzających się wartości.

```
unikalne = sorted(unikalne)
```

Należy pamiętać, że "set"jest strukturą nieuporządkowaną i nie umożliwia odwoływania się do poszczególnych elementów za pomocą indeksów, tak jak ma to miejsce w przypadku listy. To samo zadanie można jednak wykonać również w inny sposób. Przykładowo można utworzyć pustą listę i kolejno dodawać do niej tylko te elementy, które jeszcze się w niej nie znajdują:

```
unikalne = []

for liczba in sekwencja:
  if liczba not in unikalne:
    unikalne.append(liczba)
```

W tym rozwiązaniu instrukcja "not in"sprawdza, czy dana wartość nie została wcześniej dodana do listy "unikalne". Jeżeli element nie występuje jeszcze na tej liście, metoda "append()"dodaje go na jej końcu. Oba rozwiązania prowadzą do usunięcia duplikatów, mimo że wykorzystują inne mechanizmy języka Python.

4. Ponieważ zbiór "set"nie gwarantuje określonej kolejności elementów, należy uporządkować otrzymane wartości. Można to zrobić za pomocą funkcji "sorted()", która przyjmuje kolekcję elementów i zwraca nową listę zawierającą te elementy w kolejności rosnącej.

```
unikalne = sorted(unikalne)
```

W przypadku przykładowej listy:

```
[3, 5, 3, 2, 5, 7, 2, 2]
```

otrzymamy listę unikalnych i uporządkowanych wartości:

```
[2, 3, 5, 7]
```

Następnie dla każdej unikalnej wartości należy określić liczbę jej wystąpień w liście wejściowej. Można do tego wykorzystać metodę "count()", dostępną dla typu danych `list`. Metoda ta zwraca liczbę wystąpień wskazanej wartości w liście. Aby wykonać tę operację dla wszystkich elementów listy `unikalne`, wykorzystujemy pętlę `for`. W każdej iteracji zmienna `liczba` przyjmuje wartość kolejnego elementu listy.

```
for liczba in unikalne:
  ile = sekwencja.count(liczba)
  print(liczba, "wystepuje", ile, "razy")
```

Metoda "sekwencja.count(liczba)"przeszukuje listę wejściową i zwraca liczbę wystąpień aktualnie analizowanej wartości.

5. Łącząc operacje przedstawione w poprzednich krokach, można utworzyć kompletną funkcję:

```
def usun_duplikaty(sekwencja):
  unikalne = sorted(set(sekwencja))

  for liczba in unikalne:
    ile = sekwencja.count(liczba)
    print(liczba, "wystepuje", ile, "razy")
```

6. Ostatnim etapem jest przetestowanie działania funkcji. Mocno zalecane jest sprawdzenie napisanego kodu programu w celu wyszukania ewentualnych błędów w jego działaniu:

```
lista = [3, 5, 3, 2, 5, 7, 2, 2]
usun_duplikaty(lista)
```

Program powinien wypisać:

```
2 wystepuje 3 razy
3 wystepuje 2 razy
5 wystepuje 2 razy
7 wystepuje 1 razy
```

Zadania

Poniższe zadania mają na celu sprawdzenie znajomości podstaw programowania w języku Python. Zadania mogą być rozwiązywane na różne sposoby – przedstawione wymagania określają oczekiwane działanie programu, a nie sposób jego implementacji.

Zadanie 1

Napisz funkcję "convert_temp", która przyjmuje jeden argument "x"reprezentujący temperaturę w stopniach Celsjusza. Funkcja powinna sprawdzić, czy "x"jest liczbą (całkowitą lub zmiennoprzecinkową). Jeśli "x"jest liczbą, funkcja powinna przekonwertować temperaturę na stopnie Fahrenheita i wypisać wynik (Uwaga: $F = 1,8 * C + 32$).

Zadanie 2

Zdefiniuj funkcję o nazwie "usun_duplikaty", która ma jeden argument o nazwie "sekwencja"w postaci listy liczb całkowitych. Funkcja powinna wyznaczyć unikalne wartości występujące na liście, uporządkować je rosnąco, a następnie wypisać, ile razy każda z nich występuje w liście wejściowej.

Zadanie 3

Zdefiniuj anonimową funkcję za pomocą słowa kluczowego **Lambda** o nazwie "znajdz_pierwszy", która ma dwa argumenty zwane "sekwencja"i "pierwszy". Funkcja powinna sprawdzić, czy dany ciąg „sekwencja” zaczyna się od znaku określonego przez argument "pierwszy". Funkcja powinna zwracać wartość True lub False (1 lub 0).

Zadanie 4

Napisz funkcję "liczby_pierwsze", która przyjmuje jeden argument „x”. Funkcja powinna znaleźć i wypisać wszystkie liczby pierwsze z zakresu od 1 do "x". Funkcja powinna zwrócić listę wszystkich odszukanych liczb pierwszych.

Zadanie 5

Napisz funkcję "statystyki", która przyjmuje jako argument listę liczb. Funkcja powinna wyznaczyć i zwrócić słownik zawierający następujące informacje: najmniejszą wartość, największą wartość, sumę wszystkich elementów, średnią arytmetyczną i liczbę elementów listy. Przykładowa postać zwracanego słownika:

```
{
  "min": 2,
  "max": 10,
  "suma": 30,
  "srednia": 6.0,
  "liczba": 5
}
```

Funkcja powinna również poprawnie obsłużyć przypadek przekazania pustej listy.

Zadanie 6

Dana jest lista liczb całkowitych. Korzystając z mechanizmu "list comprehension", utwórz:

1. listę zawierającą wyłącznie liczby parzyste,
2. listę zawierającą kwadraty wszystkich liczb,
3. listę zawierającą kwadraty wyłącznie liczb dodatnich,
4. listę zawierającą liczby większe od średniej wartości wszystkich elementów listy wejściowej.

Przykładowa lista wejściowa:

```
liczby = [-5, -2, 0, 1, 3, 4, 8, 10]
```

Zadanie 7

Napisz funkcję "analizuj_tekst", która przyjmuje jako argument dowolny ciąg znaków. Funkcja powinna:

1. zamienić wszystkie litery na małe,
2. usunąć podstawowe znaki interpunkcyjne,
3. podzielić tekst na pojedyncze słowa,
4. policzyć liczbę wystąpień każdego słowa,
5. zwrócić wyniki w postaci słownika.

Przykładowo dla tekstu:

```
"Python jest prosty, Python jest czytelny."
```

część otrzymanego słownika powinna mieć postać:

```
{
  "python": 2,
  "jest": 2,
  "prosty": 1,
  "czytelny": 1
}
```

Zadanie 8

Korzystając z biblioteki `NumPy`, utwórz jednowymiarową tablicę zawierającą 20 losowych liczb całkowitych z zakresu od 0 do 100. Następnie:

1. wypisz całą tablicę,
2. wyznacz jej wartość minimalną i maksymalną,
3. oblicz średnią oraz odchylenie standardowe,
4. wyznacz wszystkie elementy większe od średniej,
5. uporządkuj elementy rosnąco,
6. utwórz nową tablicę zawierającą wartości znormalizowane zgodnie ze wzorem:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}.$$

Zadanie 9

Utwórz za pomocą biblioteki `NumPy` dwie macierze:

$$A \in \mathbb{R}^{3\times 4}, \qquad B \in \mathbb{R}^{4\times 2}.$$

Wartości elementów macierzy mogą zostać wybrane samodzielnie lub wygenerowane losowo. Następnie:

1. wypisz rozmiary obu macierzy,
2. wykonaj mnożenie macierzowe $A \cdot B$,
3. wypisz otrzymaną macierz oraz jej rozmiar,
4. wyznacz macierz transponowaną A^T,
5. oblicz sumę elementów w każdym wierszu macierzy A,
6. oblicz średnią wartość elementów w każdej kolumnie macierzy A.

Zadanie 10

Utwórz klasę 'Calculator' i wykonaj następujące czynności:

1. Funkcja "__init__()"powinna przyjmować argumenty "x"i "y". Domyślne wartości zarówno "x"i "y"są równe 0.
2. Utwórz funkcję klasy "Calculator"o nazwie "set". Funkcja powinna przyjmować "x"i "y"jako argumenty i zmieniać wartości self.x i self.y.
3. Utwórz funkcję klasy "Calculator"o nazwie "get". Funkcja nie ma argumentów i powinna zwracać wartości self.x oraz self.y.
4. Utwórz funkcję klasy "Calculator"o nazwie "add". Funkcja nie ma argumentów i powinna zwrócić sumę self.x i self.y.
5. Utwórz funkcję klasy "Calculator"o nazwie "multiply". Funkcja nie ma argumentów i powinna zwrócić wynik mnożenia self.x*self.y.
6. Utwórz funkcję klasy "Calculator"o nazwie "divide". Funkcja nie ma argumentów i powinna zwrócić wynik podziału self.x/self.y. Jeśli wartość self.y wynosi 0, to funkcja powinna wypisać "Błąd: dzielenie przez zero"i zwrócić None.

Zadanie 11

Utwórz klasę "Morse_translator"

1. Funkcja "__init__()"powinna przyjmować jeden argument "sequence"w postaci ciągu znaków (string) to przetłumaczenia na język Morse'a.
2. Utwórz funkcję klasy "Morse_translator"o nazwie "set". Funkcja powinna przyjąć "sequence"jako argument i zmienić wartość self.sequence.
3. Utwórz funkcję klasy "Morse_translator"o nazwie "get". Funkcja nie ma argumentów i powinna zwracać wartości self.sequence.
4. Utwórz funkcję klasy "Morse_translator"o nazwie "only_small_letters". Funkcja nie ma argumentów i powinna usunąć z self.sequence wszystkie elementy niebędące literami, a następnie zamienić wszystkie wielkie litery na małe litery.
5. Utwórz funkcję klasy „Morse_translator" o nazwie "translate". Funkcja nie ma argumentów i powinna przetłumaczyć ciąg self.sequence, po wykonaniu funkcji z funkcją „self.only_small_letters", na język Morse'a. Funkcja ma wypisać i zwrócić przetłumaczony kod Morse'a odpowiadający ciągowi znaków self.sequence.